

LB2: Integrationstesting

1 Ausgangslage

Unser Unternehmen nutzt eine Open-Source-Softwarelösung, die unseren geschäftlichen Anforderungen optimal entspricht. Die durchgeführten Unit-Tests haben bestätigt, dass die Software hält, was sie verspricht. Eine für uns essenzielle Funktionalität wurde unter Anwendung des Test-Driven-Development von uns entwickelt.

Nun steht die Überprüfung an, ob die Software, einschliesslich unserer Erweiterungen, reibungslos mit anderen Komponenten und Daten in unserer Softwarelandschaft interagiert.

2 Aufgabenstellung

2.1 Produkt

Sie schreiben ein Testkonzept. Diese senden Sie der Lehrperson am sechsten Unterrichtstag.

2.2 Sozialform

Die Bearbeitung dieses Auftrags erfolgt in Partnerarbeit.

2.3 Arbeitsauftrag

Testobjekt

Sie arbeiten weiter mit der Software aus LB1 oder Sie wählen eine neue Software. Die zu testende Software muss folgende Bedingungen erfüllen:

- Anbindung an einer Datenbank mit mindestens 3 Tabellen
- UI ist nicht zwingend

Sie testen die Integration zwischen dem Programmcode und der Datenbank.

Beispiele für Integrationskomponenten:

- Datenbankverbindung
- Datenintegrität
- Fehlerbehandlung
- Transaktionsprüfung
- Datenvalidierung
- Sicherheitsüberprüfung

Produkt

Sie verfassen ein Testkonzept für die Integration.

Sie beschreiben die Testumgebung.

Sie wählen drei Schwerpunkte (siehe Beispiele) aus und begründen Ihre Auswahl.

Sie beschreiben, was und wie genau getestet werden soll.

Zusätzlich führen Sie die Tests durch und dokumentieren die Ergebnisse.

E-Mail

Das fertige Dokument ist im PDF-Format und heisst
"M450_LB2_<Klasse>_<Name1>_<Name2>.pdf"

Sie senden die Dokumentation per E-Mail an die Lehrperson und CC an Ihren Teampartner mit Betreff "M450 LB2 <Klasse>".

3 Bewertung

3.1 Abgabezeitpunkt

Sie können die Dokumentation abgeben, sobald diese parat ist spätestens jedoch am Ende der Lektion in der sechsten Quartalswoche.

3.2 Kriterien

Siehe separates Bewertungsraster.

Pro Integrationskomponente sind 3 bis 5 Tests zu definieren.

Für eine verspätete Abgabe gibt es einen Abzug von 0.4 Note pro 24 Stunden.

3.3 Nachvollziehbarkeit bei Integrationstesting

Die nachvollziehbare Dokumentation von Integrationstests ist entscheidend für die Qualitätssicherung, die Zusammenarbeit im Team und die Reproduzierbarkeit von Testergebnissen. Hier sind einige bewährte Ansätze:

3.3.1 Detaillierte Testfälle:

Testumgebung: Dokumentieren Sie die Hardware-, Software- und Netzwerkumgebung, in der die Tests ausgeführt werden.

Klare Beschreibung: Jeder Testfall sollte eine präzise Beschreibung enthalten, was getestet wird und welche Schritte durchgeführt werden. Eventuell fügen Sie Beispiele für Eingabedaten und erwartete Ausgaben hinzu

Erwartete Ergebnisse: Definieren Sie klar, welches Ergebnis Sie erwarten.

Vorbedingungen: Definieren Sie alle Voraussetzungen, die für die Ausführung des Tests erfüllt sein müssen (z.B. bestimmte Systemkonfigurationen, vorbereitete Daten).

Nachbedingungen: Beschreiben Sie, was nach dem Test überprüft werden soll (z.B. Zustand der Datenbank, Log-Einträge).

3.3.2 Testprotokolle:

Ergebnisse: Dokumentieren Sie für jeden Testfall das tatsächliche Ergebnis, ob der Test bestanden oder fehlgeschlagen ist und gegebenenfalls Fehlermeldungen.

Screenshots/Logs: Fügen Sie Screenshots von relevanten Benutzeroberflächen oder Log-Auszüge bei, um die Ergebnisse zu veranschaulichen.

3.3.3 Warum ist eine gute Dokumentation wichtig?

Nachvollziehbarkeit: Andere Teammitglieder können die Tests besser verstehen und nachvollziehen.

Reproduzierbarkeit: Tests können leichter wiederholt werden.

Fehleranalyse: Fehler können schneller identifiziert und behoben werden.

Regressionstests: Ändern sich die Anforderungen, können die Tests leicht angepasst werden.

Auditing: Die Dokumentation dient als Nachweis für die Qualitätssicherung.

3.3.4 Zusätzliche Tipps:

Visuelle Elemente: Verwenden Sie Diagramme und Flusscharts, um komplexe Zusammenhänge zu veranschaulichen.

Wiederverwendbarkeit: Gestalten Sie die Tests und Dokumentation so, dass sie wiederverwendbar sind.

Lesbarkeit: Verwenden Sie eine klare und einfache Sprache, um die Dokumentation für alle Beteiligten verständlich zu machen.